Первое высшее техническое учебное заведение России
Санкт-Петербургский горный университет императрицы Екатерины II



Основы конструирования измерительных систем

Лекция 5. Основы объектно-ориентированного программирования

М.Г. Выстрчил, к.т.н., доцент каф. Маркшейдерского дела

г. Санкт-Петербург
2025

ГОРНЫЙ

Объектно-ориентированное программирование

Объе́ктно ориенти́рованное программи́рование (сокр. ООП) — методология или стиль программирования на основе описания типов/моделей предметной области и их взаимодействия, представленных порождением из прототипов или как экземпляры классов, которые образуют иерархию наследования.

Идеологически, ООП — подход к программированию как к моделированию информационных объектов, решающий на более высоком абстрактном уровне основную задачу структурного программирования — структурирование информации с точки зрения управляемости. Это позволяет управлять самим процессом моделирования и реализовывать крупные программные проекты.

Управляемость для иерархических систем предполагает минимизацию избыточности данных (аналогичную нормализации) и их целостность, поэтому созданное удобно управляемым — будет и удобно пониматься. Таким образом, через тактическую задачу управляемости решается стратегическая задача — транслировать понимание задачи программистом в наиболее удобную для дальнейшего использования форму.

Объектно-ориентированное программирование

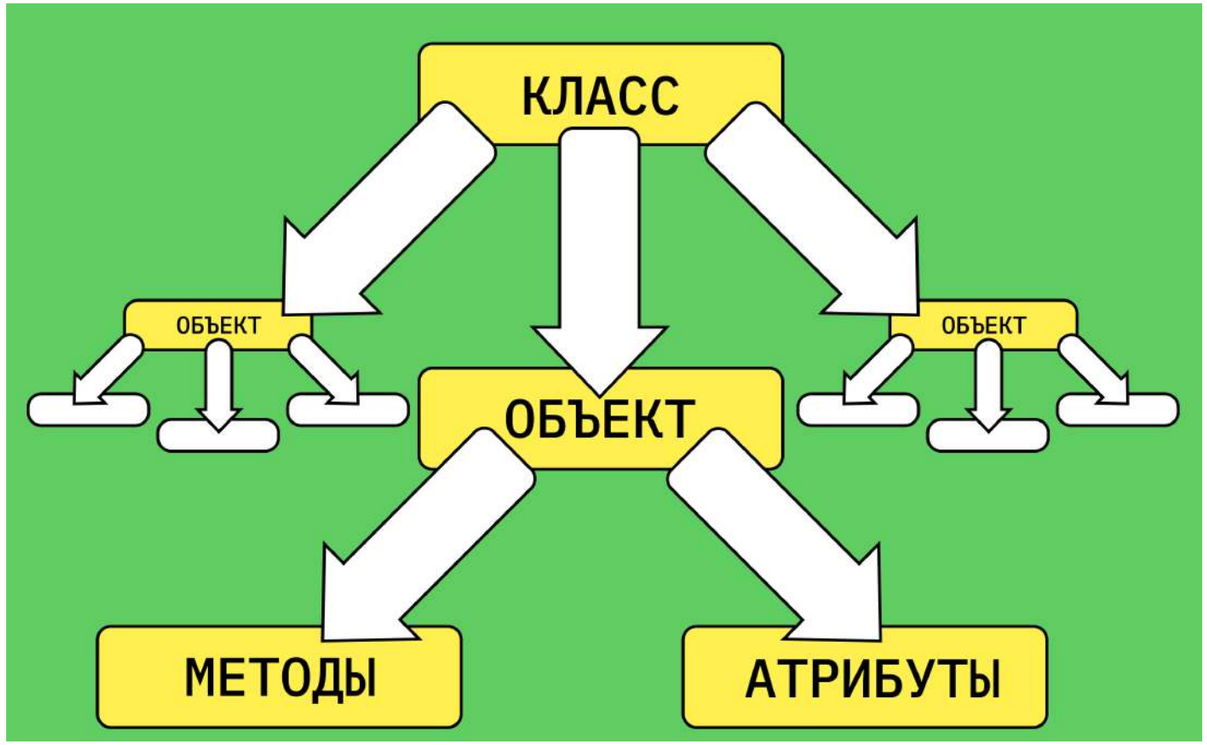
Объектно-ориентированное программирование - это способ написания кода, где программа строится из объектов.

Объект — это как бы маленькая самостоятельная "штука" в программе, которая может содержать:

- данные - **атрибуты** (например, имя пользователя)

и

- действия - **методы** (например, функция, которая выводит имя на экран).



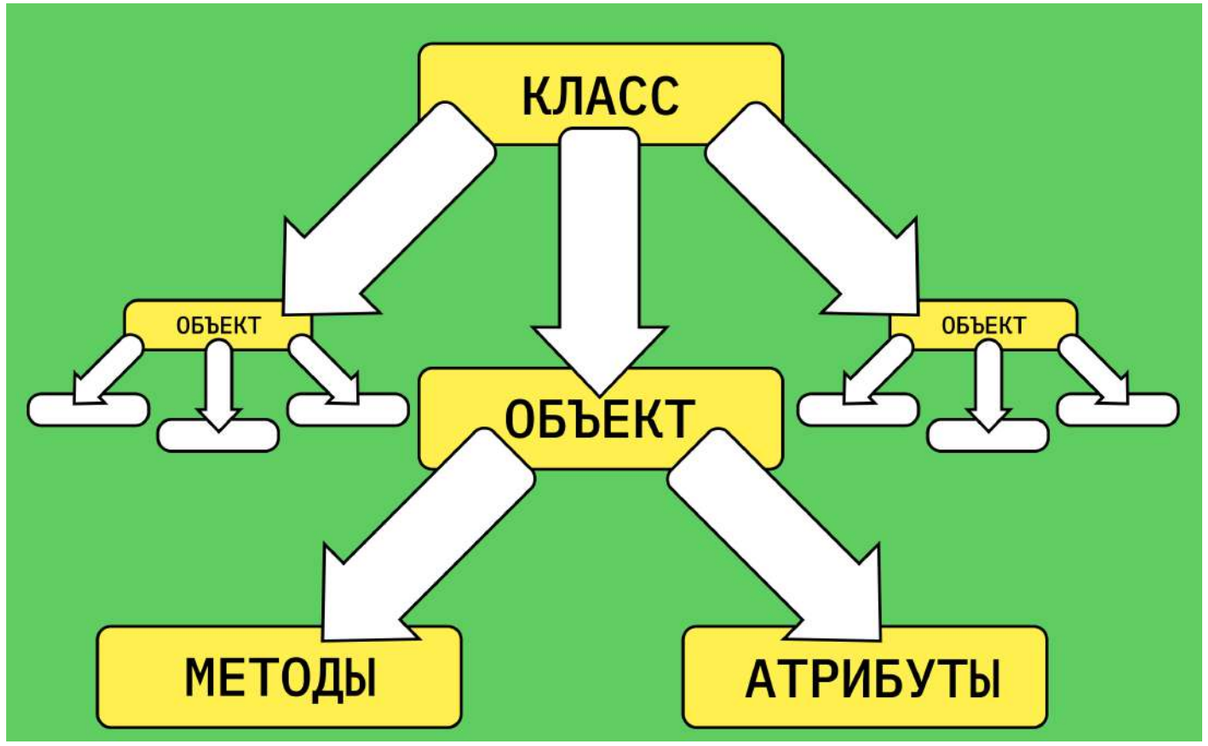
ГОРНЫЙ

Основные идеи ООП

Классы и объекты:

Класс — это как "шаблон" или "описание" для создания объектов. Например, класс "Автомобиль" может описывать, что у автомобиля есть марка, цвет и функция "ехать".

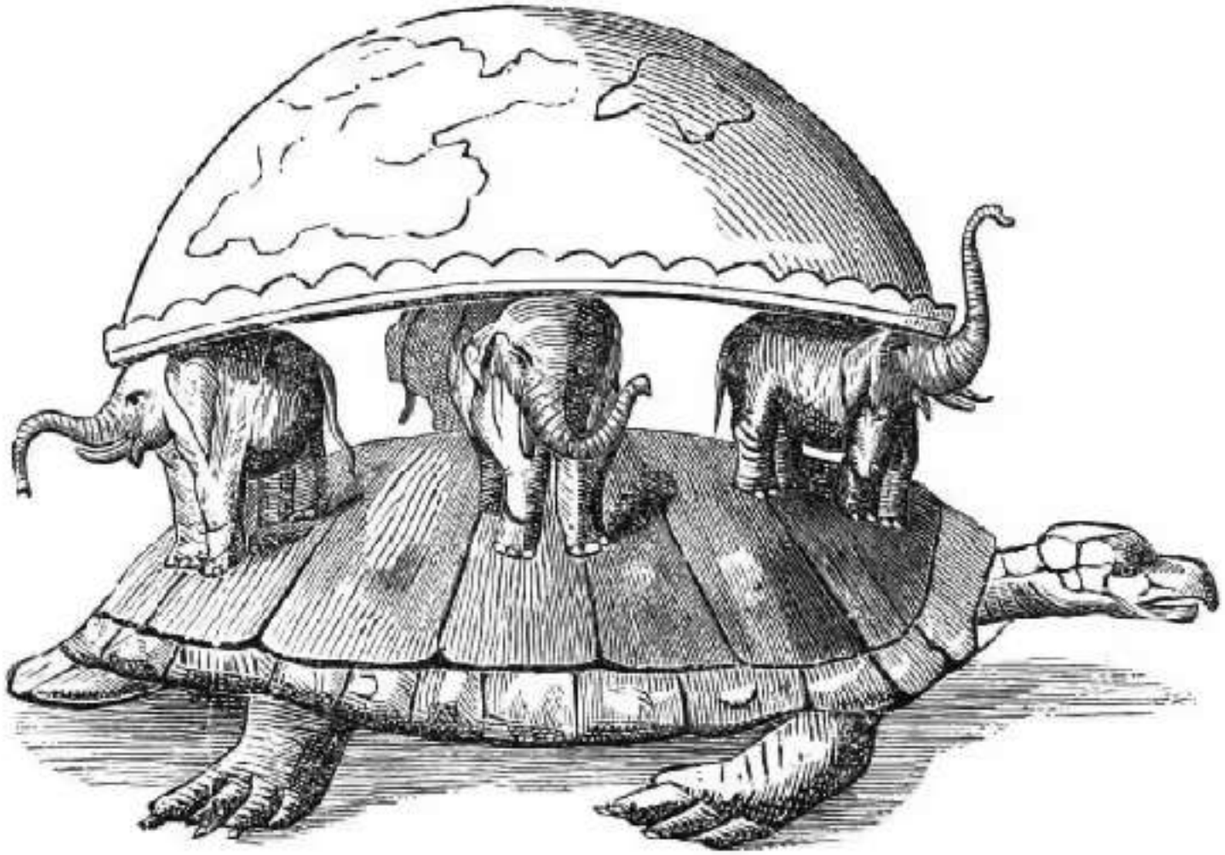
Объект — это конкретный экземпляр класса. Например, "Тойота красного цвета" — это объект класса "Автомобиль".



ГОРНЫЙ

Основные идеи ООП

- Абстракция
- Инкапсуляция
- Наследование
- Полиморфизм



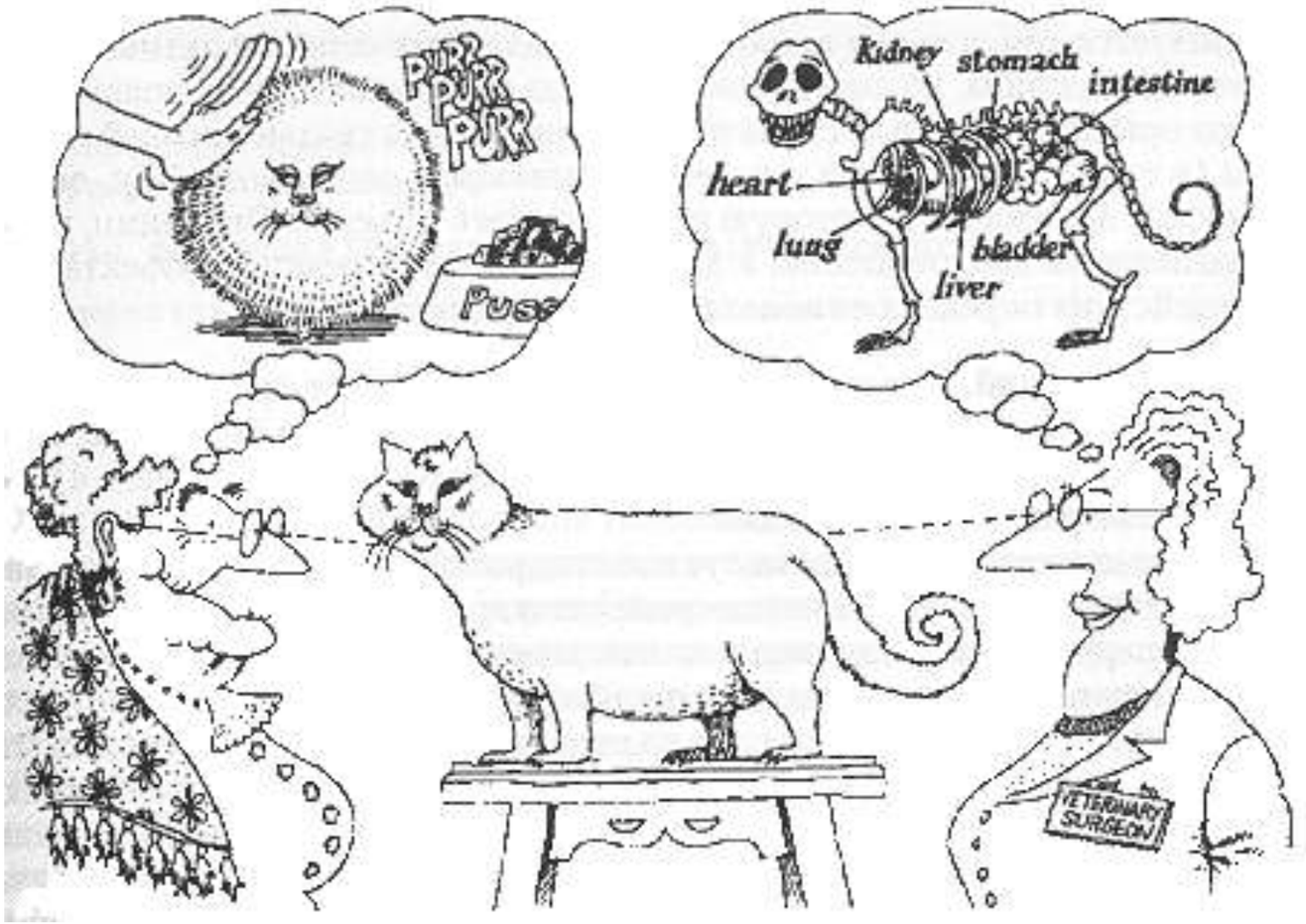
ГОРНЫЙ

Основные идеи ООП

Абстракция:

• **Что это?** Скрытие сложности и показ только необходимой информации

• **Зачем?** Упрощает использование сложных систем, позволяя работать с ними на высоком уровне



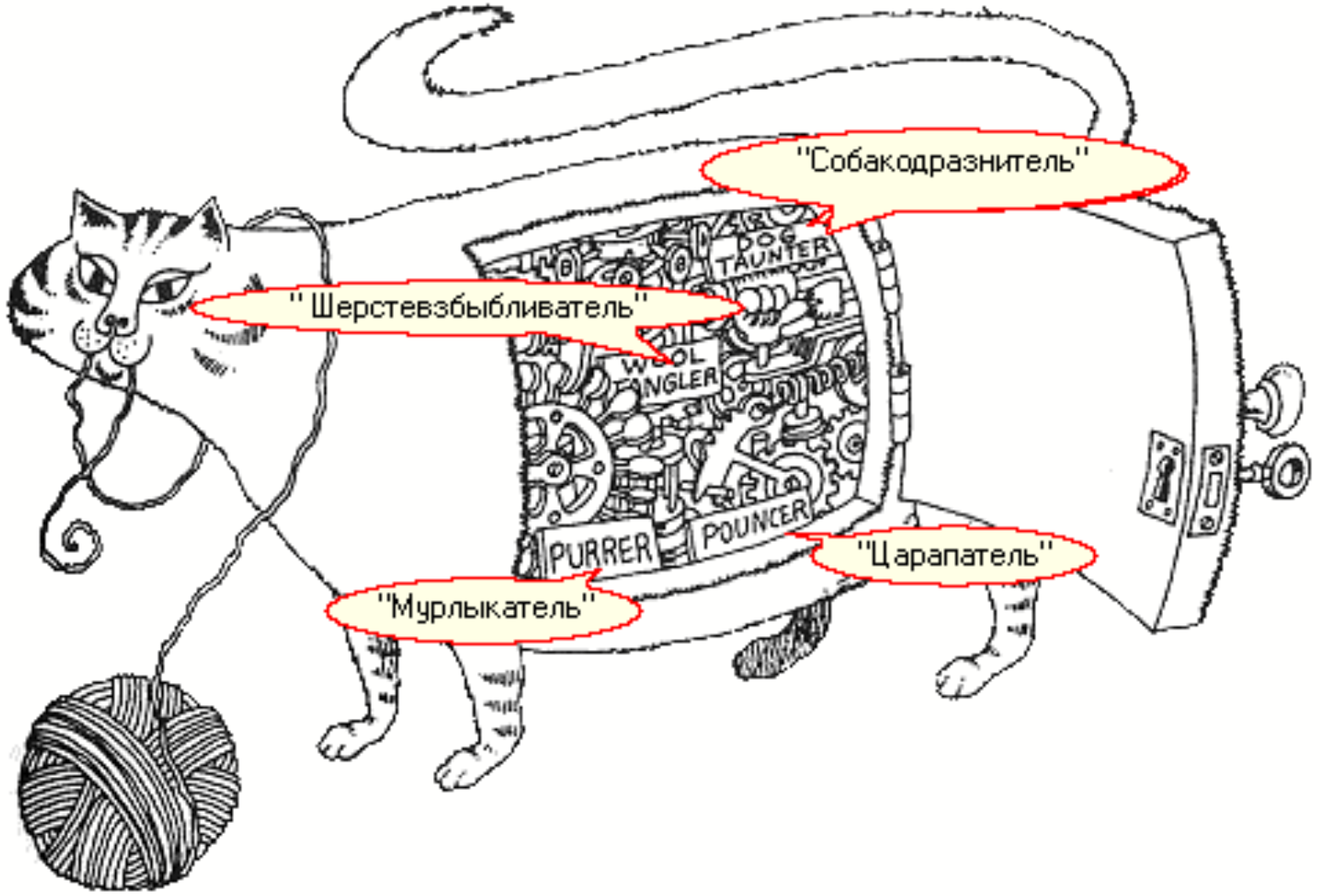
ГОРНЫЙ

Основные идеи ООП

Инкапсуляция:

• **Что это?** Скрытие внутренних деталей объекта и предоставление доступа только через определенные методы

• **Зачем?** Защита данных и контроль над их изменением, повышение безопасности и надежности кода

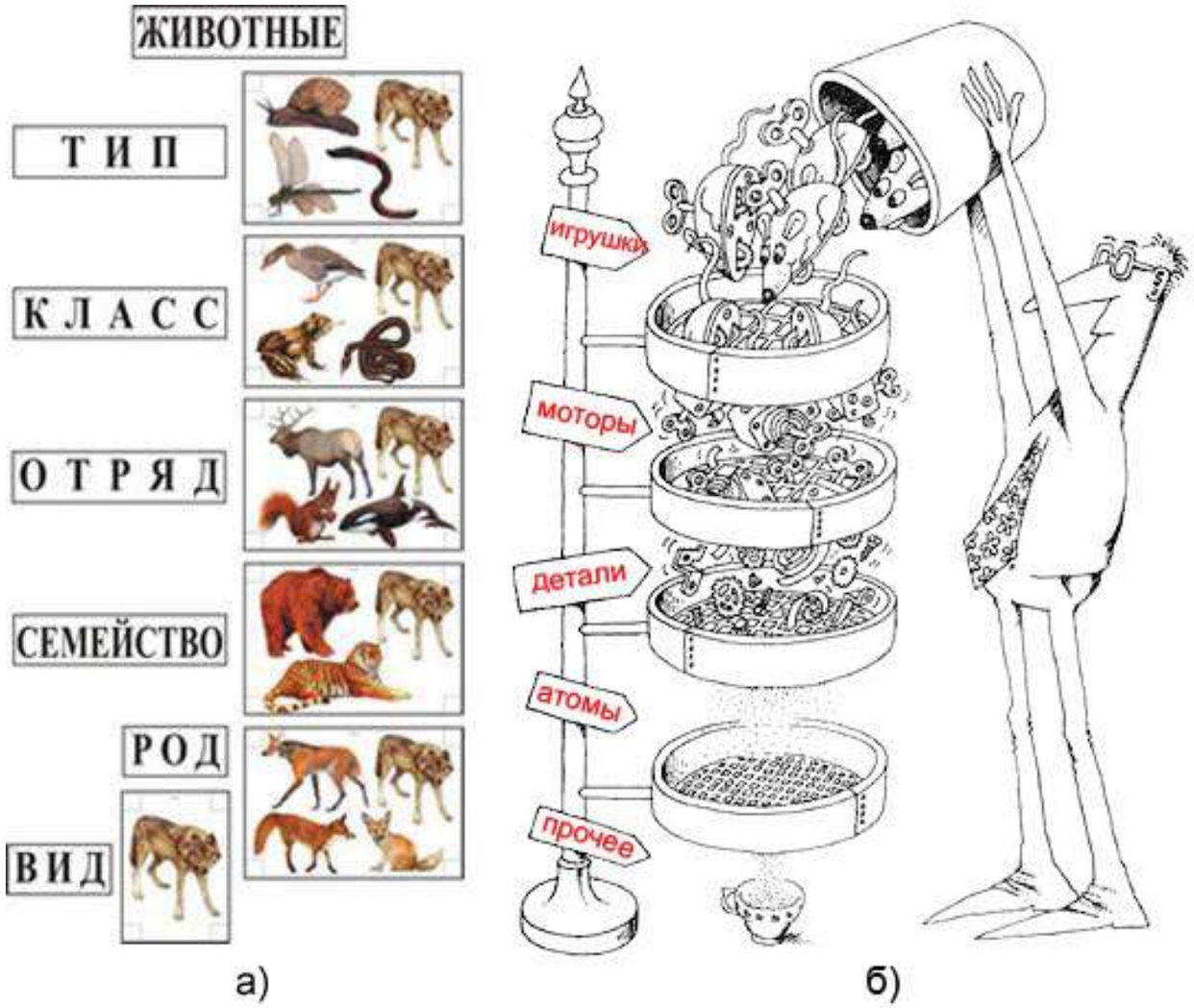


Основные идеи ООП

Наследование:

• **Что это?** Создание нового класса на основе уже существующего

• **Зачем?** Повторное использование кода и создание иерархии классов

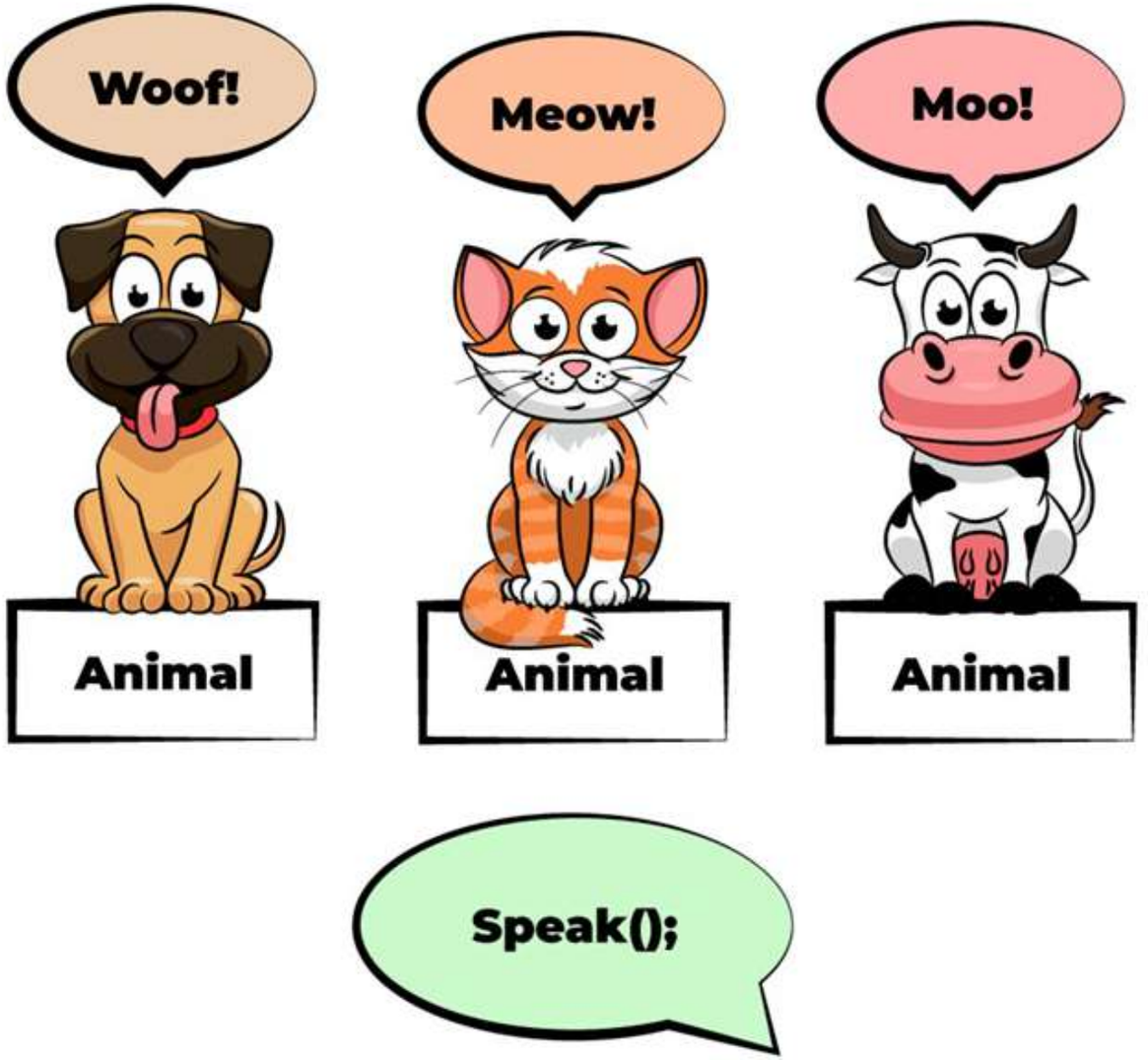


Основные идеи ООП

Полиморфизм:

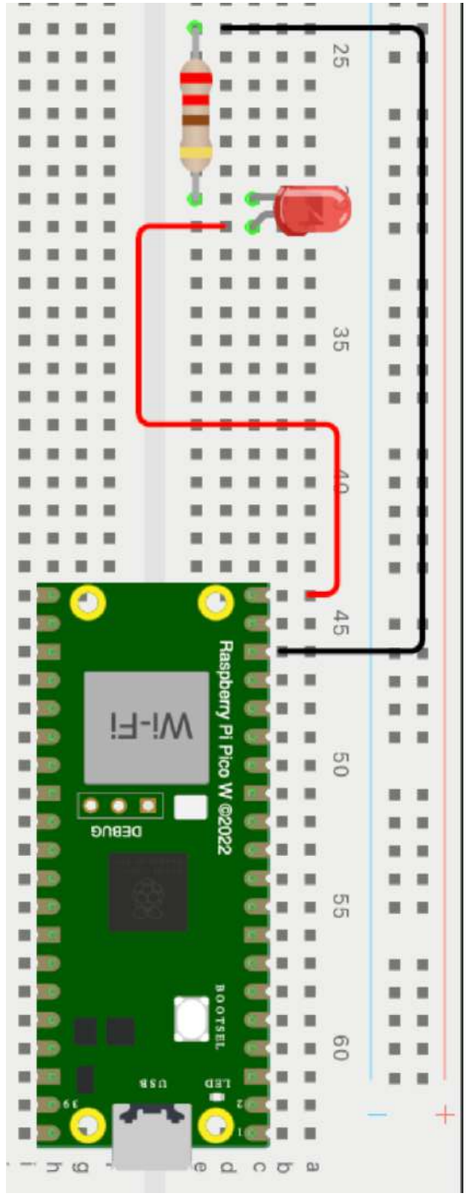
• **Что это?** Возможность использовать один интерфейс для разных классов

• **Зачем?** Гибкость и упрощение кода, позволяя одному методу работать с разными типами данных



ГОРНЫЙ

Работа со светодиодом




```
from machine import Pin
import time

led_pin = Pin(15, Pin.OUT)

while True:
    led_pin.on()
    time.sleep(3)
    led_pin.off()
    time.sleep(1)
```

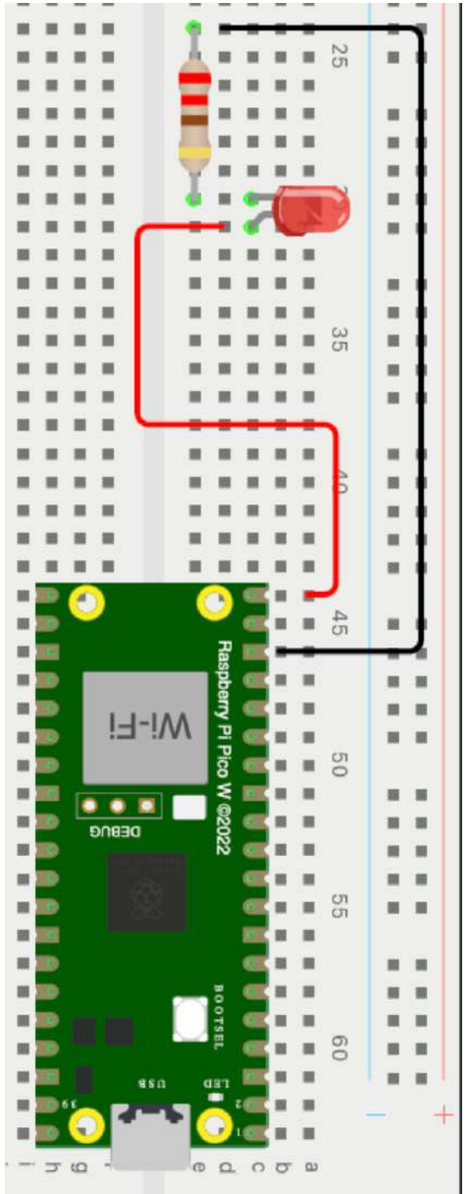
```
while True:
    led_pin.high()
    time.sleep(3)
    led_pin.low()
    time.sleep(1)
```

```
while True:
    led_pin.value(1)
    time.sleep(3)
    led_pin.value(0)
    time.sleep(1)
```

```
counter = 0
while True:
    led_pin.value(counter % 2)
    time.sleep(3)
    counter += 1
```

ГОРНЫЙ

Класс для светодиода




```
from time import sleep
from machine import Pin

class Led:
    def __init__(self, led_pin):
        self.led = Pin(led_pin, Pin.OUT)

    def on(self):
        self.led.on()

    def off(self):
        self.led.off()


led = Led(15)
while True:
    led.on()
    sleep(3)
    led.off()
    sleep(1)
```

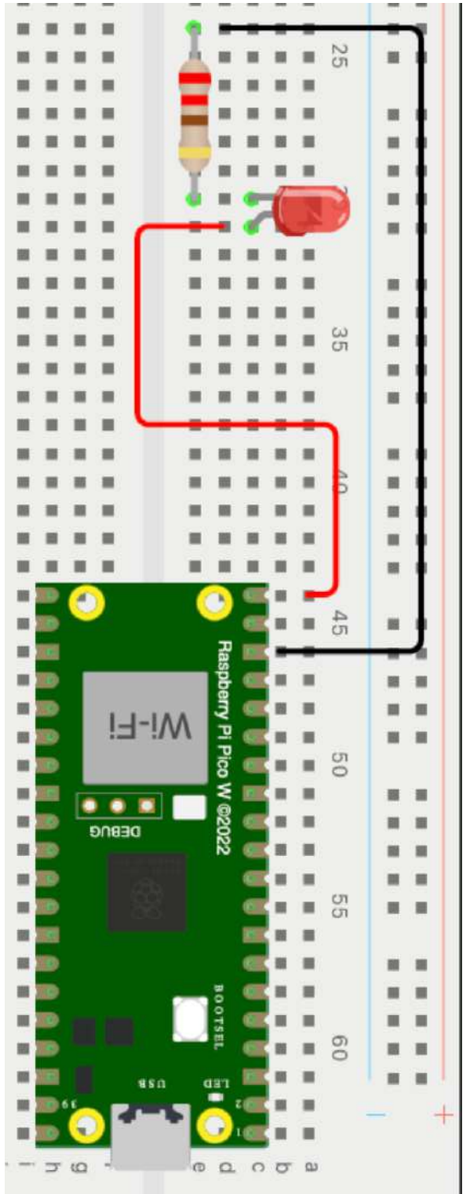
```
from machine import Pin
import time

led_pin = Pin(15, Pin.OUT)

while True:
    led_pin.on()
    time.sleep(3)
    led_pin.off()
    time.sleep(1)
```

ГОРНЫЙ

Расширяем функциональность класса Led




```
from time import sleep
from machine import Pin

class Led:
    def __init__(self, led_pin):
        self.led = Pin(led_pin, Pin.OUT)

    def on(self):
        self.led.on()

    def off(self):
        self.led.off()


led = Led(15)
while True:
    led.on()
    sleep(3)
    led.off()
    sleep(1)
```

```
from time import sleep
from machine import Pin, PWM


class Led:
    def __init__(self, led_pin):
        self.led = PWM(Pin(led_pin, Pin.OUT), freq=1000)

    def on(self):
        self.led.duty_u16(65535)

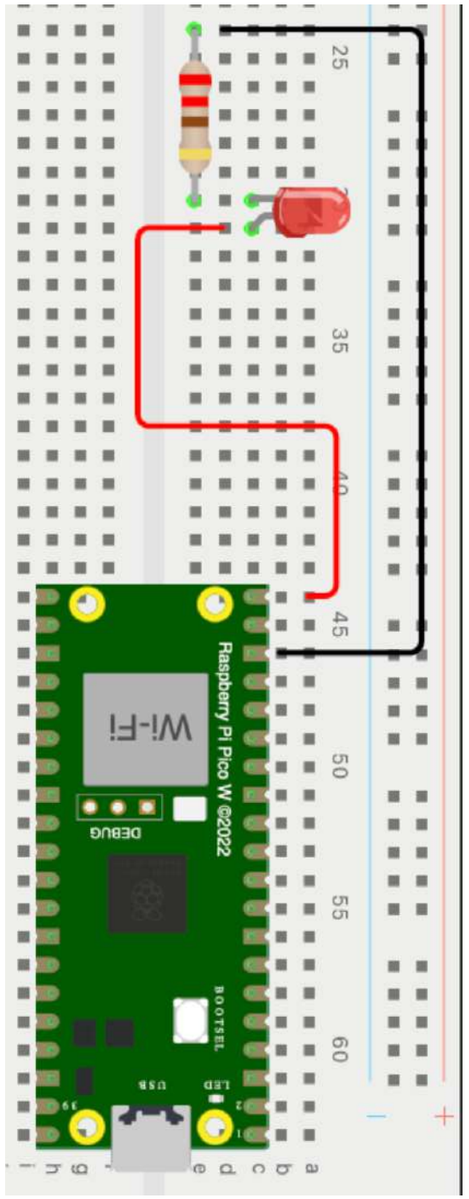
    def off(self):
        self.led.duty_u16(0)

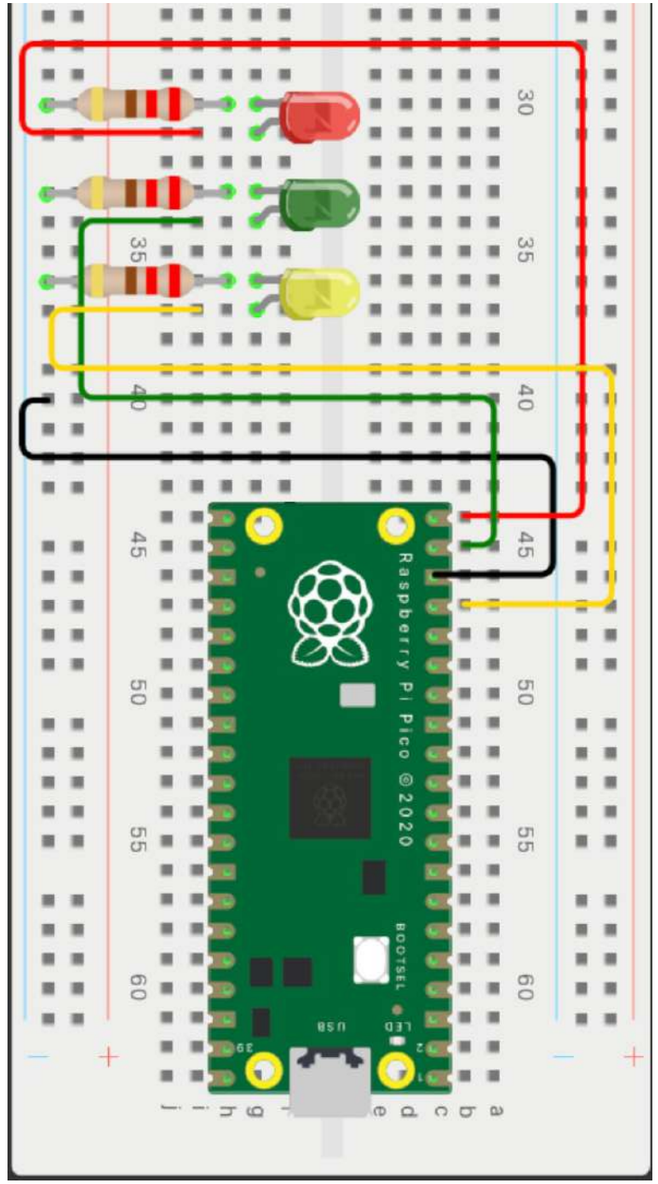
    def smooth_switching_on(self):
        for duty in range(65535):
            self.led.duty_u16(duty)
            sleep(0.0001)

    def smooth_switching_off(self):
        for duty in range(65535, -1, -1):
            self.led.duty_u16(duty)
            sleep(0.0001)


led = Led(15)
while True:
    led.smooth_switching_on()
    sleep(3)
    led.off()
    sleep(1)
```

Расширяем дальше и осознаем свою ошибку






```python
from time import sleep
from machine import Pin, PWM

class Led:
    def __init__(self, led_pin):
        self.led = PWM(Pin(led_pin, Pin.OUT), freq=1000)

    def on(self):
        self.led.duty_u16(65535)

    def off(self):
        self.led.duty_u16(0)

    def smooth_switching_on(self):   ...

    def smooth_switching_off(self):   ...

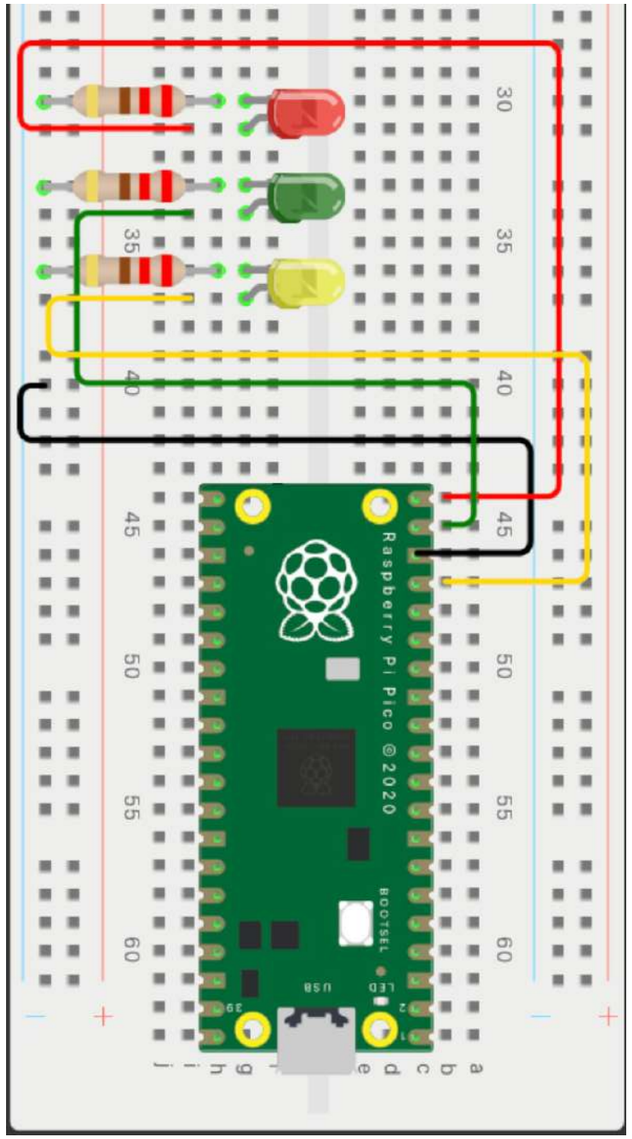
    def blink(self, sleep_time_in_sec):
        self.on()
        sleep(sleep_time_in_sec)
        self.off()
        sleep(sleep_time_in_sec)

red_led = Led(15)
green_led = Led(14)
yellow_led = Led(13)


while True:
    red_led.blink(2)
    # green_led.blink(1)
```

ГОРНЫЙ

Рефакторим код и устраняем конфликт задержек




```
from time import ticks_ms
from machine import Pin, PWM

class Led:

    def __init__(self, led_pin):
        self.led = PWM(Pin(led_pin, Pin.OUT), freq=1000)
        self.timer = ticks_ms()
        self.state = "OFF"

    def on(self):
        self.led.duty_u16(65535)
        self.state = "ON"

    def off(self):
        self.led.duty_u16(0)
        self.state = "OFF"

    def blink(self, blink_time_in_sec):
        current_time = ticks_ms()
        if (current_time - self.timer) / 1000 >= blink_time_in_sec:
            self.timer = current_time
            if self.state == "ON":
                self.off()
            else:
                self.on()
```

```
red_led = Led(15)
green_led = Led(14)
yellow_led = Led(13)

while True:
    red_led.blink(1)
    green_led.blink(2)
    yellow_led.blink(0.5)
```

ГОРНЫЙ

Оцениваем результат

```python
from time import ticks_ms
from machine import Pin, PWM

class Led:
    BRIGHTNESS_SCALE = 100

    def __init__(self, led_pin):
        self.led = PWM(Pin(led_pin, Pin.OUT), freq=1000)
        self.timer = ticks_ms()
        self.brightness = 0
        self.state = "OFF"

    def on(self):
        self.led.duty_u16(65535)
        self.state = "ON"

    def off(self):
        self.led.duty_u16(0)
        self.state = "OFF"

    def turn_to_brightness(self, brightness):
        if brightness <= 0:
            brightness = 0
        elif brightness > self.BRIGHTNESS_SCALE:
            brightness = self.BRIGHTNESS_SCALE
        duty = int(65535 / self.BRIGHTNESS_SCALE * brightness)
        self.led.duty_u16(duty)
        self.brightness = brightness
```

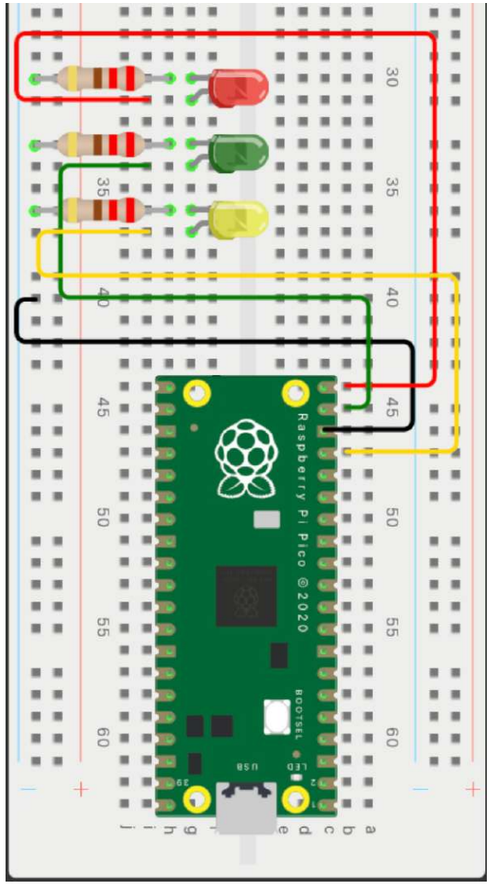
```python
    def smooth_switching_on(self, switching_time=1):
        sleep_time = switching_time / self.BRIGHTNESS_SCALE
        if self.state == "STOP":
            self.state = "UP"
        if self.brightness == self.BRIGHTNESS_SCALE:
            self.state = "STOP"
        if (ticks_ms() - self.timer) / 1000 >= sleep_time and self.state
== "UP":
            self.timer = ticks_ms()
            self.brightness += 1
            self.turn_to_brightness(self.brightness)

    def smooth_switching_off(self, switching_time=1):
        sleep_time = switching_time / self.BRIGHTNESS_SCALE
        if self.state == "STOP":
            self.state = "DOWN"
        if self.brightness == 0:
            self.state = "STOP"
        if (ticks_ms() - self.timer) / 1000 >= sleep_time and self.state
== "DOWN":
            self.timer = ticks_ms()
            self.brightness -= 1
            self.turn_to_brightness(self.brightness)

    def blink(self, blink_time_in_sec):
        if (ticks_ms() - self.timer) / 1000 >= blink_time_in_sec:
            self.timer = ticks_ms()
            if self.state != "OFF":
                self.off()
            else:
                self.on()
```

ГОРНЫЙ

Проверяем его




```
from time import ticks_ms
from machine import Pin, PWM

class Led:
    BRIGHTNESS_SCALE = 100

    > def __init__(self, led_pin):    …

    > def on(self):    …

    > def off(self):    …

    > def turn_to_brightness(self, brightness):    …

    > def smooth_switching_on(self, switching_time=1):    …

    > def smooth_switching_off(self, switching_time=1):

    > def blink(self, blink_time_in_sec):    …
```

```
if __name__ == "__main__":
    red_led = Led(15)
    green_led = Led(14)
    yellow_led = Led(13)

    while True:
        red_led.smooth_switching_on(1)
        red_led.smooth_switching_off(0)
        green_led.smooth_switching_on(1)
        green_led.smooth_switching_off(0.25)

        yellow_led.blink(1)
```

ГОРНЫЙ

Контрольные вопросы

- Что такое ООП?
- Чем класс отличается от объекта?
- Назовите четыре базовых принципа ООП?
- Что такое абстракция?
- Что такое атрибут?
- Чем статический атрибут отличается от атрибута объекта?
- Что такое метод, чем он отличается от функции?
- Зачем нужно ООП?